

Cross-Domain State Preservation Functor

Jinwook Kim (Jay)

March 27, 2026

Abstract

We present a generic, reusable framework for verifying that state transitions are faithfully preserved when synchronized across heterogeneous domains (blockchains, off-chain ledgers, etc.).

The core abstraction is a hierarchy of Isabelle/HOL locales: `state_machine` defines finite-state deterministic transition systems with terminal states; `state_preservation` captures the naturality (homomorphism) condition ensuring that a synchronization map commutes with transitions; `symmetric_state_preservation` extends this to bidirectional synchronization with inverse roundtrip guarantees; and `multi_domain_preservation` generalizes to N domains sharing the same abstract state machine, with a cross-domain consistency theorem.

As a practical application, we instantiate this framework with the regulatory state model of Oraclizer, a state synchronization oracle for tokenized assets across blockchain networks and off-chain ledgers. The model comprises five states (ACTIVE, FROZEN, SEIZED, CONFISCATED, RESTRICTED) and seven actions. The instantiation proves that a synchronization protocol with preemptive locking preserves cross-chain regulatory consistency (the *regulatory homomorphism* theorem), that the global validity invariant is maintained across synchronization events (*valid state preservation*), and that the generic multi-domain locale applies parametrically to the regulatory model.

The design is inspired by Lochbihler’s Merkle Functor (AFP entry `ADS_Functor`), which abstracted authenticated data structures into composable building blocks. This entry extends that pattern to a different axis of abstraction: cross-domain state preservation targeting synchronization correctness rather than data structure integrity.

Contents

1	Overview	1
1.1	Design Decisions	2
2	Generic State Machine Locale	2
3	Cross-Domain State Preservation Locale	3

4	Symmetric State Preservation	6
5	Multi-Domain State Preservation	7
6	Regulatory State and Action Definitions	9
7	Fundamental Properties of the Transition Function	10
7.1	Invariant I1: CONFISCATED is terminal	10
7.2	Invariant I2: CONFISCATE reaches CONFISCATED from any non-terminal state	10
7.3	Invariant I3: Determinism (inherent from function definition)	10
7.4	SEIZED to FROZEN exclusion	10
7.5	FROZEN to RESTRICTED exclusion	11
7.6	Transition completeness: every non-terminal state has at least one valid action	11
7.7	Reachability from ACTIVE	11
7.8	Return paths to ACTIVE	12
8	State Machine Instance	12
9	Asset and Global State Modeling	13
10	State Accessors and Predicates	14
11	Global State Validity	14
12	Synchronization Protocol	15
12.1	Lock acquisition	15
12.2	State update across chains	15
12.3	The synchronization function	16
13	Synchronization Properties	16
13.1	Idempotency	16
13.2	Sync isolation: other assets are not affected	16
13.3	Sync correctness: source chain updated	17
13.4	Properties of update_all_chains	17
13.5	Lock mutual exclusion	18
14	Main Theorem: Cross-Chain Regulatory Consistency	19
15	Valid State Preservation	21
15.1	Part 1: consistent_state preservation	21
15.2	Part 2: no_locked_without_reason preservation	24
15.3	The valid_state preservation theorem	25
16	Multi-Domain Locale Instantiation	25

1 Overview

This AFP entry consists of two theory files:

State_Preservation.thy defines domain-independent locales for state machines, pairwise state preservation (with naturality), symmetric bidirectional preservation, and multi-domain preservation. All locales are parameterized over abstract types and can be instantiated with any domain that has finite states, deterministic transitions, and optional terminal states.

Regulatory_Instance.thy instantiates the generic framework with a concrete regulatory state model. It defines the five regulatory states and seven regulatory actions, proves fundamental properties (terminal absorption, universal reachability of confiscation, transition exclusions with legal justification), models a synchronization protocol with preemptive locking, and proves the main theorems: *regulatory homomorphism* (cross-chain consistency after sync), *valid state preservation* (sync maintains the global validity invariant), and *multi-domain parametric instantiation*.

1.1 Design Decisions

The regulatory state model incorporates several deliberate design decisions based on legal precedence and operational considerations:

- RECOVER and LIQUIDATE are excluded from the state machine as they involve force transfers or external DEX interactions rather than state transitions.
- The direct transition SEIZED $\rightarrow$ FROZEN is excluded because seizure is a strictly stronger legal constraint than freezing; relaxation requires explicit release first.
- The direct transition FROZEN $\rightarrow$ RESTRICTED is excluded to enforce passage through ACTIVE.
- Preemptive locking prevents concurrent regulatory actions on the same asset, *not* double-spending (which is handled at a different layer).

```
theory State-Preservation
  imports Main
begin
```

2 Generic State Machine Locale

A state machine with finite state set, deterministic partial transition function, and optional terminal states. Terminal states accept no transitions.

```

locale state-machine =
  fixes states :: 's set
    and actions :: 'a set
    and transition :: 's  $\Rightarrow$  'a  $\Rightarrow$  's option
    and terminal :: 's set
  assumes finite-states: finite states
    and finite-actions: finite actions
    and terminal-subset: terminal  $\subseteq$  states
    and terminal-absorbing:  $\llbracket s \in \text{terminal}; a \in \text{actions} \rrbracket \Longrightarrow \text{transition } s \ a = \text{None}$ 
  and transition-closed:  $\llbracket s \in \text{states}; a \in \text{actions}; \text{transition } s \ a = \text{Some } s' \rrbracket \Longrightarrow s' \in \text{states}$ 
  and transition-domain:  $\llbracket s \notin \text{states} \rrbracket \Longrightarrow \text{transition } s \ a = \text{None}$ 
begin

```

Determinism is inherent: transition is a function, not a relation.

lemma transition-deterministic:

$\text{transition } s \ a = \text{Some } s1 \Longrightarrow \text{transition } s \ a = \text{Some } s2 \Longrightarrow s1 = s2$

by simp

Sequential composition of actions (partial, returns None on first failure).

fun apply-actions :: 's $\Rightarrow$ 'a list $\Rightarrow$'s option **where**

$\text{apply-actions } s \ [] = \text{Some } s$

| $\text{apply-actions } s \ (a \ \# \ as) = (\text{case } \text{transition } s \ a \ \text{of}$

$\text{None} \Rightarrow \text{None}$

| $\text{Some } s' \Rightarrow \text{apply-actions } s' \ as)$

lemma apply-actions-closed:

$\llbracket s \in \text{states}; \forall a \in \text{set } as. a \in \text{actions}; \text{apply-actions } s \ as = \text{Some } s' \rrbracket \Longrightarrow s' \in \text{states}$

by (induction as arbitrary: s) (auto split: option.splits dest: transition-closed)

Note: $\text{apply-actions } ?s \ [] = \text{Some } ?s$ already provides the simp rule $\text{apply-actions } s \ [] = \text{Some } s$, so no separate [simp] lemma is needed.

lemma apply-actions-terminal:

$\llbracket s \in \text{terminal}; a \in \text{actions} \rrbracket \Longrightarrow \text{apply-actions } s \ (a \ \# \ as) = \text{None}$

by (simp add: terminal-absorbing)

end

3 Cross-Domain State Preservation Locale

Two state machines connected by a synchronization map. The key property (naturality / homomorphism) states that synchronizing after a local

transition yields the same result as transitioning after synchronization.

More precisely: for state machines (S_src, T_src) and (S_tgt, T_tgt) connected by sync (mapping source states to target states), the following diagram commutes:

$$\begin{array}{ccc} s_src & \xrightarrow{T_src(a)} & s_src' \\ \downarrow \text{sync} & & \downarrow \text{sync} \\ s_tgt & \xrightarrow{T_tgt(a)} & s_tgt' \end{array}$$

That is: $\text{sync } (T_src \ s \ a) = T_tgt \ (\text{sync } s) \ a$

This is the structural preservation guarantee: a transition performed at the source domain, when synchronized, produces the same result as performing the corresponding transition at the target domain.

locale *state-preservation* =

source: state-machine $states_s \ actions_s \ transition_s \ terminal_s$ +

target: state-machine $states_t \ actions_t \ transition_t \ terminal_t$

for $states_s :: 's \text{ set}$ **and** $actions_s :: 'a \text{ set}$

and $transition_s :: 's \Rightarrow 'a \Rightarrow 's \text{ option}$

and $terminal_s :: 's \text{ set}$

and $states_t :: 't \text{ set}$ **and** $actions_t :: 'b \text{ set}$

and $transition_t :: 't \Rightarrow 'b \Rightarrow 't \text{ option}$

and $terminal_t :: 't \text{ set}$ +

fixes $state\text{-}map :: 's \Rightarrow 't$

and $action\text{-}map :: 'a \Rightarrow 'b$

assumes *state-map-well-defined*: $s \in states_s \implies state\text{-}map \ s \in states_t$

and *action-map-well-defined*: $a \in actions_s \implies action\text{-}map \ a \in actions_t$

and *terminal-preservation*: $s \in terminal_s \implies state\text{-}map \ s \in terminal_t$

— The naturality / homomorphism condition:

and *naturality*:

$\llbracket s \in states_s; a \in actions_s; transition_s \ s \ a = Some \ s' \rrbracket$

$\implies transition_t \ (state\text{-}map \ s) \ (action\text{-}map \ a) = Some \ (state\text{-}map \ s')$

and *naturality-none*:

$\llbracket s \in states_s; a \in actions_s; transition_s \ s \ a = None \rrbracket$

$\implies transition_t \ (state\text{-}map \ s) \ (action\text{-}map \ a) = None$

begin

The fundamental theorem: state preservation extends to sequences of actions. If a sequence of transitions is valid at the source, the mapped sequence is valid at the target with the mapped final state.

theorem *sequential-preservation*:

assumes $s \in states_s$

and $\forall a \in as. a \in actions_s$

and $source.apply\text{-}actions \ s \ as = Some \ s'$

shows $target.apply\text{-}actions \ (state\text{-}map \ s) \ (map \ action\text{-}map \ as) = Some \ (state\text{-}map \ s')$

using *assms*

proof (*induction as arbitrary: s*)

case *Nil*

then show *?case* **by** *simp*

next

case (*Cons a as*)

```

then obtain  $s\text{-mid}$  where
  step:  $\text{transition}_s\ s\ a = \text{Some}\ s\text{-mid}$  and
  rest:  $\text{source.apply-actions}\ s\text{-mid}\ as = \text{Some}\ s'$ 
  by (auto split: option.splits)
from Cons.premis(1) Cons.premis(2) step have  $s\text{-mid} \in \text{states}_s$ 
  using source.transition-closed by auto
from Cons.premis(1) Cons.premis(2) step
  have mapped-step:  $\text{transition}_t\ (\text{state-map}\ s)\ (\text{action-map}\ a) = \text{Some}\ (\text{state-map}\ s\text{-mid})$ 
  using naturality by auto
from  $\langle s\text{-mid} \in \text{states}_s \rangle$  Cons.premis(2) rest
  have target.apply-actions  $(\text{state-map}\ s\text{-mid})\ (\text{map}\ \text{action-map}\ as) = \text{Some}\ (\text{state-map}\ s')$ 
  using Cons.IH by auto
with mapped-step show ?case by simp
qed

theorem sequential-preservation-none:
  assumes  $s \in \text{states}_s$ 
  and  $\forall a \in \text{set}\ as.\ a \in \text{actions}_s$ 
  and  $\text{source.apply-actions}\ s\ as = \text{None}$ 
  shows  $\text{target.apply-actions}\ (\text{state-map}\ s)\ (\text{map}\ \text{action-map}\ as) = \text{None}$ 
  using assms
proof (induction as arbitrary: s)
  case Nil
  then show ?case by simp
next
  case (Cons a as)
  show ?case
  proof (cases  $\text{transition}_s\ s\ a$ )
    case None
    then have  $\text{transition}_t\ (\text{state-map}\ s)\ (\text{action-map}\ a) = \text{None}$ 
      using naturality-none Cons.premis(1) Cons.premis(2) by auto
    then show ?thesis by simp
  next
    case (Some  $s\text{-mid}$ )
    then have mid-in:  $s\text{-mid} \in \text{states}_s$ 
      using source.transition-closed Cons.premis(1) Cons.premis(2) by auto
    from Some Cons.premis(3) have  $\text{source.apply-actions}\ s\text{-mid}\ as = \text{None}$ 
      by simp
    then have ih:  $\text{target.apply-actions}\ (\text{state-map}\ s\text{-mid})\ (\text{map}\ \text{action-map}\ as) = \text{None}$ 
      using Cons.IH mid-in Cons.premis(2) by auto
    from Some have  $\text{transition}_t\ (\text{state-map}\ s)\ (\text{action-map}\ a) = \text{Some}\ (\text{state-map}\ s\text{-mid})$ 
      using naturality Cons.premis(1) Cons.premis(2) by auto
    with ih show ?thesis by simp
  qed
qed

```

Terminal states are preserved: if a source state is terminal, its image is terminal.

corollary *terminal-image-absorbing*:

assumes $s \in \text{terminal}_s$ **and** $a \in \text{actions}_s$

shows $\text{transition}_t (\text{state-map } s) (\text{action-map } a) = \text{None}$

proof –

from *assms(1)* **have** $s \in \text{states}_s$ **using** *source.terminal-subset* **by** *auto*

from *assms* **have** $\text{transition}_s s a = \text{None}$

using *source.terminal-absorbing* **by** *auto*

then show *?thesis*

using *naturality-none* $\langle s \in \text{states}_s \rangle$ *assms(2)* **by** *auto*

qed

end

4 Symmetric State Preservation

For bidirectional synchronization, both directions must preserve structure. This locale composes two **state_preservation** instances with inverse maps, establishing that synchronization in either direction is structure-preserving.

locale *symmetric-state-preservation* =

forward: state-preservation

$\text{states}_s \text{ actions}_s \text{ transition}_s \text{ terminal}_s$

$\text{states}_t \text{ actions}_t \text{ transition}_t \text{ terminal}_t$

$\text{state-map} \text{ action-map} +$

backward: state-preservation

$\text{states}_t \text{ actions}_t \text{ transition}_t \text{ terminal}_t$

$\text{states}_s \text{ actions}_s \text{ transition}_s \text{ terminal}_s$

$\text{state-map-inv} \text{ action-map-inv}$

for $\text{states}_s :: 's \text{ set}$ **and** $\text{actions}_s :: 'a \text{ set}$

and $\text{transition}_s :: 's \Rightarrow 'a \Rightarrow 's \text{ option}$

and $\text{terminal}_s :: 's \text{ set}$

and $\text{states}_t :: 't \text{ set}$ **and** $\text{actions}_t :: 'b \text{ set}$

and $\text{transition}_t :: 't \Rightarrow 'b \Rightarrow 't \text{ option}$

and $\text{terminal}_t :: 't \text{ set}$

and $\text{state-map} :: 's \Rightarrow 't$ **and** $\text{action-map} :: 'a \Rightarrow 'b$

and $\text{state-map-inv} :: 't \Rightarrow 's$ **and** $\text{action-map-inv} :: 'b \Rightarrow 'a +$

assumes *roundtrip-state-src*: $s \in \text{states}_s \implies \text{state-map-inv} (\text{state-map } s) = s$

and *roundtrip-state-tgt*: $t \in \text{states}_t \implies \text{state-map} (\text{state-map-inv } t) = t$

and *roundtrip-action-src*: $a \in \text{actions}_s \implies \text{action-map-inv} (\text{action-map } a) = a$

and *roundtrip-action-tgt*: $b \in \text{actions}_t \implies \text{action-map} (\text{action-map-inv } b) = b$

begin

Under symmetric preservation with roundtrip maps, **state_map** is injective on the state set – distinct source states map to distinct target states. This guarantees no information loss during synchronization.

lemma *state-map-injective*:

```

     $\llbracket s1 \in \text{states}_s; s2 \in \text{states}_s; \text{state-map } s1 = \text{state-map } s2 \rrbracket \implies s1 = s2$ 
    using roundtrip-state-src by metis
end

```

5 Multi-Domain State Preservation

Generalization to N domains sharing the same abstract state machine. This models scenarios where a single asset exists on multiple domains (e.g., blockchain networks), all of which must reflect the same state.

The key property: if one domain transitions, the synchronization brings ALL other domains to the same resulting state.

We model this as: all domains are instances of the same abstract state machine, connected by identity-like state/action maps. The "same abstract state machine" assumption holds when the protocol enforces a uniform state model across all domains.

```

locale multi-domain-preservation =
  fixes domains :: 'd set
  and states :: 's set
  and actions :: 'a set
  and transition :: 's  $\Rightarrow$  'a  $\Rightarrow$  's option
  and terminal :: 's set
  and domain-state :: 'd  $\Rightarrow$  'id  $\Rightarrow$  's option
    — Current state of asset 'id in domain 'd
  assumes fin-domains: finite domains
  and sm: state-machine states actions transition terminal
  and consistent-init:
    — Precondition: all domains holding an asset agree on its state
     $\llbracket d1 \in \text{domains}; d2 \in \text{domains};$ 
      domain-state d1 aid = Some s1; domain-state d2 aid = Some s2  $\rrbracket$ 
     $\implies s1 = s2$ 
begin

```

Which domains hold a given asset.

```

definition connected-domains :: 'id  $\Rightarrow$  'd set where
  connected-domains aid = {d  $\in$  domains. domain-state d aid  $\neq$  None}

```

The global consensus state of an asset (well-defined by `consistent_init`).

```

definition consensus-state :: 'id  $\Rightarrow$  's option where
  consensus-state aid =
    (if connected-domains aid = {} then None
     else domain-state (SOME d. d  $\in$  connected-domains aid) aid)

```

After synchronizing an action on an asset, all connected domains reflect the new state. This is the cross-domain consistency theorem.

```

definition sync-all ::
  'd  $\Rightarrow$  'a  $\Rightarrow$  'id  $\Rightarrow$  ('d  $\Rightarrow$  'id  $\Rightarrow$  's option)  $\Rightarrow$  ('d  $\Rightarrow$  'id  $\Rightarrow$  's option) option

```


where

```

sync-all source action aid ds =
  (case ds source aid of
    None  $\Rightarrow$  None
  | Some s  $\Rightarrow$ 
    (case transition s action of
      None  $\Rightarrow$  None
    | Some s'  $\Rightarrow$ 
      Some ( $\lambda d$  aid'.
        if aid' = aid  $\wedge d \in$  connected-domains aid
        then Some s'
        else ds d aid'))))

```

The cross-domain consistency theorem: after **sync_all**, every connected domain agrees on the new state.

theorem *cross-domain-consistency*:

```

assumes source  $\in$  domains
and domain-state source aid = Some s
and s  $\in$  states
and action  $\in$  actions
and transition s action = Some s'
and sync-all source action aid domain-state = Some ds'
and d  $\in$  connected-domains aid
shows ds' d aid = Some s'
proof –
from assms(2,5,6) have
  ds' = ( $\lambda d$  aid'.
    if aid' = aid  $\wedge d \in$  connected-domains aid
    then Some s'
    else domain-state d aid')
unfolding sync-all-def by auto
with assms(7) show ?thesis by simp
qed

```

sync_all does not affect other assets.

theorem *sync-isolation*:

```

assumes sync-all source action aid domain-state = Some ds'
and aid'  $\neq$  aid
shows ds' d aid' = domain-state d aid'
proof –
from assms(1) obtain s s' where
  domain-state source aid = Some s
  transition s action = Some s'
unfolding sync-all-def by (auto split: option.splits)
then have ds' = ( $\lambda d$  aid'.
  if aid' = aid  $\wedge d \in$  connected-domains aid
  then Some s'
  else domain-state d aid')
using assms(1) unfolding sync-all-def by auto

```

```

    with assms(2) show ?thesis by auto
qed

end

end

```

```

theory Regulatory-Instance
  imports State-Preservation
begin

```

6 Regulatory State and Action Definitions

```

datatype reg-state = ACTIVE | FROZEN | SEIZED | CONFISCATED | RE-
STRICTED

```

```

datatype reg-action = FREEZE | SEIZE | CONFISCATE | RESTRICT
| UNFREEZE | UNRESTRICT | RELEASE

```

The regulatory transition function. Partial: returns None for invalid (state, action) combinations. The transition table encodes legal precedence: seizure is stronger than freezing, confiscation is terminal and universally reachable, and intermediate states must return to Active before lateral transitions.

```

fun reg-transition :: reg-state  $\Rightarrow$  reg-action  $\Rightarrow$  reg-state option where
  — From ACTIVE: all forward actions valid
  | reg-transition ACTIVE FREEZE      = Some FROZEN
  | reg-transition ACTIVE SEIZE       = Some SEIZED
  | reg-transition ACTIVE CONFISCATE = Some CONFISCATED
  | reg-transition ACTIVE RESTRICT   = Some RESTRICTED
  | reg-transition ACTIVE UNFREEZE   = None
  | reg-transition ACTIVE UNRESTRICT = None
  | reg-transition ACTIVE RELEASE    = None
  — From FROZEN: unfreeze, escalate to SEIZED or CONFISCATED
  | reg-transition FROZEN UNFREEZE   = Some ACTIVE
  | reg-transition FROZEN SEIZE      = Some SEIZED
  | reg-transition FROZEN CONFISCATE = Some CONFISCATED
  | reg-transition FROZEN FREEZE     = None
  | reg-transition FROZEN RESTRICT   = None
  | reg-transition FROZEN UNRESTRICT = None
  | reg-transition FROZEN RELEASE    = None
  — From SEIZED: release or final confiscation only
  | reg-transition SEIZED RELEASE    = Some ACTIVE
  | reg-transition SEIZED CONFISCATE = Some CONFISCATED
  | reg-transition SEIZED FREEZE     = None
  | reg-transition SEIZED SEIZE      = None
  | reg-transition SEIZED RESTRICT   = None

```

```

| reg-transition SEIZED UNFREEZE    = None
| reg-transition SEIZED UNRESTRICT = None
| — From CONFISCATED: terminal no transitions out
| reg-transition CONFISCATED FREEZE  = None
| reg-transition CONFISCATED SEIZE   = None
| reg-transition CONFISCATED CONFISCATE = None
| reg-transition CONFISCATED RESTRICT  = None
| reg-transition CONFISCATED UNFREEZE  = None
| reg-transition CONFISCATED UNRESTRICT = None
| reg-transition CONFISCATED RELEASE  = None
| — From RESTRICTED: unrestrict, escalate to FROZEN or CONFISCATED
| reg-transition RESTRICTED UNRESTRICT = Some ACTIVE
| reg-transition RESTRICTED FREEZE     = Some FROZEN
| reg-transition RESTRICTED CONFISCATE = Some CONFISCATED
| reg-transition RESTRICTED RESTRICT  = None
| reg-transition RESTRICTED SEIZE      = None
| reg-transition RESTRICTED UNFREEZE  = None
| reg-transition RESTRICTED RELEASE   = None

```

7 Fundamental Properties of the Transition Function

7.1 Invariant I1: CONFISCATED is terminal

lemma *confiscated-terminal*:
 reg-transition CONFISCATED a = None
 by (cases a) auto

7.2 Invariant I2: CONFISCATE reaches CONFISCATED from any non-terminal state

lemma *confiscate-universal*:
 $s \neq \text{CONFISCATED} \implies \text{reg-transition } s \text{ CONFISCATE} = \text{Some CONFISCATED}$
 by (cases s) auto

7.3 Invariant I3: Determinism (inherent from function definition)

Determinism is trivially guaranteed by the **fun** definition.

7.4 SEIZED to FROZEN exclusion

SEIZED is a strictly stronger constraint than FROZEN. Direct transition from SEIZED to FROZEN is legally nonsensical (cannot "weaken" a court-ordered seizure to a mere freeze). The path is: RELEASE then ACTIVE then FREEZE.

lemma *seized-no-freeze*:
reg-transition SEIZED FREEZE = None
by *simp*

lemma *seized-no-unfreeze*:
reg-transition SEIZED UNFREEZE = None
by *simp*

lemma *seized-no-restrict*:
reg-transition SEIZED RESTRICT = None
by *simp*

7.5 FROZEN to RESTRICTED exclusion

Must go through ACTIVE: UNFREEZE then ACTIVE then RESTRICT.

lemma *frozen-no-restrict*:
reg-transition FROZEN RESTRICT = None
by *simp*

7.6 Transition completeness: every non-terminal state has at least one valid action

lemma *non-terminal-has-action*:
assumes $s \neq \text{CONFISCATED}$
shows $\exists a. \text{reg-transition } s \ a \neq \text{None}$
proof (*cases s*)
 case *ACTIVE*
 show *?thesis*
 apply (*rule exI[where x=FREEZE]*)
 using *ACTIVE* **by** *simp*
next
 case *FROZEN*
 show *?thesis*
 apply (*rule exI[where x=UNFREEZE]*)
 using *FROZEN* **by** *simp*
next
 case *SEIZED*
 show *?thesis*
 apply (*rule exI[where x=RELEASE]*)
 using *SEIZED* **by** *simp*
next
 case *CONFISCATED*
 then show *?thesis* **using** *assms* **by** *contradiction*
next
 case *RESTRICTED*
 show *?thesis*
 apply (*rule exI[where x=UNRESTRICT]*)
 using *RESTRICTED* **by** *simp*
qed

7.7 Reachability from ACTIVE

Every state is reachable from ACTIVE via some sequence of actions.

lemma *active-reaches-frozen*: *reg-transition ACTIVE FREEZE = Some FROZEN*
by *simp*

lemma *active-reaches-seized*: *reg-transition ACTIVE SEIZE = Some SEIZED*
by *simp*

lemma *active-reaches-confiscated*: *reg-transition ACTIVE CONFISCATE = Some CONFISCATED*
by *simp*

lemma *active-reaches-restricted*: *reg-transition ACTIVE RESTRICT = Some RESTRICTED*
by *simp*

7.8 Return paths to ACTIVE

lemma *frozen-returns*: *reg-transition FROZEN UNFREEZE = Some ACTIVE*
by *simp*

lemma *seized-returns*: *reg-transition SEIZED RELEASE = Some ACTIVE*
by *simp*

lemma *restricted-returns*: *reg-transition RESTRICTED UNRESTRICT = Some ACTIVE*
by *simp*

8 State Machine Instance

We show that `reg_state` / `reg_action` / `reg_transition` satisfy the `state_machine` locale assumptions.

definition *reg-states* :: *reg-state set* **where**
reg-states = {*ACTIVE*, *FROZEN*, *SEIZED*, *CONFISCATED*, *RESTRICTED*}

definition *reg-actions* :: *reg-action set* **where**
reg-actions = {*FREEZE*, *SEIZE*, *CONFISCATE*, *RESTRICT*, *UNFREEZE*, *UNRESTRICT*, *RELEASE*}

definition *reg-terminal* :: *reg-state set* **where**
reg-terminal = {*CONFISCATED*}

lemma *reg-states-UNIV*: *reg-states* = *UNIV*
unfolding *reg-states-def* **by** (*auto*, *case-tac x*, *auto*)

lemma *reg-actions-UNIV*: *reg-actions* = *UNIV*
unfolding *reg-actions-def* **by** (*auto*, *case-tac x*, *auto*)

```

lemma reg-transition-closed:
   $\llbracket \text{reg-transition } s \ a = \text{Some } s' \rrbracket \implies s' \in \text{reg-states}$ 
  unfolding reg-states-def by (cases s; cases a; auto)

interpretation reg-sm: state-machine reg-states reg-actions reg-transition reg-terminal
proof unfold-locales
  show finite reg-states unfolding reg-states-def by auto
next
  show finite reg-actions unfolding reg-actions-def by auto
next
  show  $\text{reg-terminal} \subseteq \text{reg-states}$  unfolding reg-terminal-def reg-states-def by auto
next
  fix s a
  assume  $s \in \text{reg-terminal} \ a \in \text{reg-actions}$ 
  then show  $\text{reg-transition } s \ a = \text{None}$ 
    unfolding reg-terminal-def by (auto simp: confiscated-terminal)
next
  fix s a s'
  assume  $s \in \text{reg-states} \ a \in \text{reg-actions} \ \text{reg-transition } s \ a = \text{Some } s'$ 
  then show  $s' \in \text{reg-states}$ 
    using reg-transition-closed by auto
next
  fix s :: reg-state and a :: reg-action
  assume  $s \notin \text{reg-states}$ 
  then show  $\text{reg-transition } s \ a = \text{None}$ 
    using reg-states-UNIV by auto
qed

```

9 Asset and Global State Modeling

```

type-synonym asset-id = nat
type-synonym chain-id = nat
type-synonym authority-id = nat
type-synonym timestamp = nat

record asset-state =
  as-asset-id :: asset-id
  as-reg-state :: reg-state
  as-owner    :: nat      — Abstract address
  as-locked   :: bool     — Preemptive lock

type-synonym chain-state = asset-id  $\Rightarrow$  asset-state option

record global-state =
  gs-chains    :: chain-id  $\Rightarrow$  chain-state
  gs-locks     :: asset-id  $\Rightarrow$  bool

record oss-message =

```

msg-action :: *reg-action*
msg-asset-id :: *asset-id*
msg-authority :: *authority-id*
msg-timestamp :: *timestamp*
msg-source :: *chain-id*
msg-targets :: *chain-id set*

10 State Accessors and Predicates

definition *get-asset-state* :: *global-state* $\Rightarrow$ *chain-id* $\Rightarrow$ *asset-id* $\Rightarrow$ *asset-state option* **where**

get-asset-state *gs* *cid* *aid* = *gs-chains* *gs* *cid* *aid*

definition *get-reg-state* :: *global-state* $\Rightarrow$ *chain-id* $\Rightarrow$ *asset-id* $\Rightarrow$ *reg-state option* **where**

get-reg-state *gs* *cid* *aid* =
 (case *get-asset-state* *gs* *cid* *aid* of
 None $\Rightarrow$ *None*
 | *Some ast* $\Rightarrow$ *Some* (*as-reg-state* *ast*))

definition *asset-exists* :: *global-state* $\Rightarrow$ *chain-id* $\Rightarrow$ *asset-id* $\Rightarrow$ *bool* **where**

asset-exists *gs* *cid* *aid* = (*get-asset-state* *gs* *cid* *aid* $\neq$ *None*)

definition *is-locked* :: *global-state* $\Rightarrow$ *asset-id* $\Rightarrow$ *bool* **where**

is-locked *gs* *aid* = *gs-locks* *gs* *aid*

Connected chains: all chains holding a given asset.

definition *connected-chains* :: *global-state* $\Rightarrow$ *asset-id* $\Rightarrow$ *chain-id set* **where**

connected-chains *gs* *aid* = {*cid*. *asset-exists* *gs* *cid* *aid*}

11 Global State Validity

A global state is valid if all chains holding the same asset agree on its regulatory state. This is the cross-chain consistency invariant that synchronization must preserve.

definition *consistent-state* :: *global-state* $\Rightarrow$ *bool* **where**

consistent-state *gs* $\equiv$
 $\forall$ *c1* *c2* *aid* *s1* *s2*.
get-reg-state *gs* *c1* *aid* = *Some* *s1* $\longrightarrow$
get-reg-state *gs* *c2* *aid* = *Some* *s2* $\longrightarrow$
s1 = *s2*

definition *no-locked-without-reason* :: *global-state* $\Rightarrow$ *bool* **where**

no-locked-without-reason *gs* $\equiv$
 $\forall$ *aid*. $\neg$ *is-locked* *gs* *aid*

— In a quiescent valid state, no assets are locked. Locks are transient, held only during sync operations.

definition *valid-state* :: *global-state* $\Rightarrow$ *bool* **where**
valid-state *gs* $\equiv$ *consistent-state* *gs* $\wedge$ *no-locked-without-reason* *gs*

12 Synchronization Protocol

12.1 Lock acquisition

definition *acquire-lock* :: *global-state* $\Rightarrow$ *asset-id* $\Rightarrow$ *global-state option* **where**
acquire-lock *gs* *aid* =
 (if *is-locked* *gs* *aid* then *None*
 else *Some* (*gs* $\llcorner$ *gs-locks* := (*gs-locks* *gs*)(*aid* := *True*) $\llcorner$)))

definition *release-lock* :: *global-state* $\Rightarrow$ *asset-id* $\Rightarrow$ *global-state* **where**
release-lock *gs* *aid* = *gs* $\llcorner$ *gs-locks* := (*gs-locks* *gs*)(*aid* := *False*) $\llcorner$

12.2 State update across chains

Update the regulatory state of an asset on a specific chain. Preserves all other asset fields (owner, etc.).

definition *update-chain-reg-state* ::
global-state $\Rightarrow$ *chain-id* $\Rightarrow$ *asset-id* $\Rightarrow$ *reg-state* $\Rightarrow$ *global-state*
where
update-chain-reg-state *gs* *cid* *aid* *new-st* =
 (let *old-chain* = *gs-chains* *gs* *cid* in
 let *new-chain* = (λ *aid'*.
 if *aid'* = *aid* then
 (case *old-chain* *aid* of
 None $\Rightarrow$ None
 | *Some* *ast* $\Rightarrow$ *Some* (*ast* $\llcorner$ *as-reg-state* := *new-st* $\llcorner$)))
 else *old-chain* *aid'*)
 in *gs* $\llcorner$ *gs-chains* := (*gs-chains* *gs*)(*cid* := *new-chain*) $\llcorner$))

Update all connected chains to a new regulatory state.

We define this directly rather than using `Finite_Set.fold`, because the update for a given asset on each chain is independent we simply construct a new global state where every chain's view of the asset is updated. This avoids the need for `comp_fun_commute` proofs.

definition *update-all-chains* ::
global-state $\Rightarrow$ *asset-id* $\Rightarrow$ *reg-state* $\Rightarrow$ *chain-id set* $\Rightarrow$ *global-state*
where
update-all-chains *gs* *aid* *new-st* *targets* =
gs $\llcorner$ *gs-chains* := (λ *cid*.
 if *cid* $\in$ *targets* then
 (λ *aid'*. if *aid'* = *aid* then
 (case *gs-chains* *gs* *cid* *aid* of
 None $\Rightarrow$ None
 | *Some* *ast* $\Rightarrow$ *Some* (*ast* $\llcorner$ *as-reg-state* := *new-st* $\llcorner$)))
 else *gs-chains* *gs* *cid* *aid*)


```

    else gs-chains gs cid aid')
  else gs-chains gs cid)  $\mathbb{I}$ 

```

12.3 The synchronization function

sync: the core synchronization operation.

Given a source chain, action, and **asset_id**: 1. Verify the asset exists on the source chain 2. Check the transition is valid 3. Acquire global lock (fails if already locked) 4. Update all connected chains to the new state 5. Release lock

Returns *None* if any step fails (invalid transition, lock contention, etc.)

definition *sync* ::

chain-id $\Rightarrow$ *reg-action* $\Rightarrow$ *asset-id* $\Rightarrow$ *global-state* $\Rightarrow$ *global-state option*

where

```

sync source action aid gs =
  (case get-reg-state gs source aid of
    None  $\Rightarrow$  None
  | Some current-st  $\Rightarrow$ 
    (case reg-transition current-st action of
      None  $\Rightarrow$  None
    | Some new-st  $\Rightarrow$ 
      (case acquire-lock gs aid of
        None  $\Rightarrow$  None
      | Some gs-locked  $\Rightarrow$ 
        let targets = connected-chains gs aid in
        let gs-updated = update-all-chains gs-locked aid new-st targets in
        Some (release-lock gs-updated aid))))

```

13 Synchronization Properties

13.1 Idempotency

After synchronization, applying the same action again yields *None* (the state has already transitioned, so the same transition from the new state is typically invalid) or is a no-op.

More precisely: if *sync* succeeds and produces **new_st**, then **reg_transition new_st action** = *None* for most action/state pairs (the transition table has no self-loops).

lemma *no-self-loops*:

```

reg-transition s a = Some s  $\implies$  False
by (cases s; cases a; auto)

```

lemma *transition-changes-state*:

```

reg-transition s a = Some s'  $\implies$  s  $\neq$  s'
using no-self-loops by auto

```

13.2 Sync isolation: other assets are not affected

Synchronization on asset *aid* does not change the state of any other asset (where the asset identifier differs) on any chain.

```
lemma update-chain-other-asset:  
  assumes aid' ≠ aid  
  shows gs-chains (update-chain-reg-state gs cid aid new-st) cid' aid' =  
        gs-chains gs cid' aid'  
proof (cases cid' = cid)  
  case True  
    then show ?thesis  
      unfolding update-chain-reg-state-def Let-def  
      using assms by auto  
next  
  case False  
    then show ?thesis  
      unfolding update-chain-reg-state-def Let-def by auto  
qed
```

13.3 Sync correctness: source chain updated

After *update_chain_reg_state*, the target chain reflects the new state.

```
lemma update-chain-same-asset:  
  assumes gs-chains gs cid aid = Some ast  
  shows gs-chains (update-chain-reg-state gs cid aid new-st) cid aid =  
        Some (ast | as-reg-state := new-st |)  
  unfolding update-chain-reg-state-def Let-def  
  using assms by auto
```

13.4 Properties of *update_all_chains*

update_all_chains correctly updates any chain in the target set.

```
lemma update-all-chains-in-targets:  
  assumes cid ∈ targets  
    and gs-chains gs cid aid = Some ast  
  shows gs-chains (update-all-chains gs aid new-st targets) cid aid =  
        Some (ast | as-reg-state := new-st |)  
  unfolding update-all-chains-def using assms by auto
```

```
lemma update-all-chains-in-targets-none:  
  assumes cid ∈ targets  
    and gs-chains gs cid aid = None  
  shows gs-chains (update-all-chains gs aid new-st targets) cid aid = None  
  unfolding update-all-chains-def using assms by auto
```

```
lemma update-all-chains-outside-targets:  
  assumes cid ∉ targets  
  shows gs-chains (update-all-chains gs aid new-st targets) cid = gs-chains gs cid
```

unfolding *update-all-chains-def* **using** *assms* **by** *auto*

lemma *update-all-chains-other-asset*:

assumes $aid' \neq aid$

shows $gs-chains (update-all-chains\ gs\ aid\ new-st\ targets)\ cid\ aid' =$
 $gs-chains\ gs\ cid\ aid'$

unfolding *update-all-chains-def* **using** *assms* **by** *auto*

lemma *update-all-chains-locks*:

$gs-locks (update-all-chains\ gs\ aid\ new-st\ targets) = gs-locks\ gs$

unfolding *update-all-chains-def* **by** *auto*

Key lemma: `get_reg_state` after `update_all_chains` for a chain in targets.

lemma *update-all-chains-reg-state*:

assumes $cid \in targets$

and $get-asset-state\ gs\ cid\ aid = Some\ ast$

shows $get-reg-state (update-all-chains\ gs\ aid\ new-st\ targets)\ cid\ aid = Some\ new-st$

proof –

from *assms*(2) **have** $chain: gs-chains\ gs\ cid\ aid = Some\ ast$

unfolding *get-asset-state-def* **by** *simp*

then have $gs-chains (update-all-chains\ gs\ aid\ new-st\ targets)\ cid\ aid =$
 $Some (ast \lfloor as-reg-state := new-st \rfloor)$

using *update-all-chains-in-targets[OF assms(1)]* **by** *simp*

then show *?thesis*

unfolding *get-reg-state-def* *get-asset-state-def* **by** *simp*

qed

13.5 Lock mutual exclusion

If a lock is held, concurrent sync on the same asset fails. This prevents concurrent regulatory action conflicts.

lemma *lock-mutual-exclusion*:

assumes $is-locked\ gs\ aid$

shows $acquire-lock\ gs\ aid = None$

unfolding *acquire-lock-def* **using** *assms* **by** *auto*

lemma *lock-acquire-success*:

assumes $\neg is-locked\ gs\ aid$

shows $\exists gs'. acquire-lock\ gs\ aid = Some\ gs' \wedge is-locked\ gs'\ aid$

proof –

from *assms* **have** $\neg gs-locks\ gs\ aid$

unfolding *is-locked-def* **by** *simp*

then have $acquire-lock\ gs\ aid = Some (gs \lfloor gs-locks := (gs-locks\ gs)(aid := True) \rfloor)$

$\rfloor)$

unfolding *acquire-lock-def* *is-locked-def* **by** *simp*

moreover have $is-locked (gs \lfloor gs-locks := (gs-locks\ gs)(aid := True) \rfloor) aid$

unfolding *is-locked-def* **by** *simp*

ultimately show *?thesis* by auto
qed

lemma *lock-release*:
shows $\neg$ *is-locked* (*release-lock* *gs* *aid*) *aid*
unfolding *release-lock-def is-locked-def* by auto

14 Main Theorem: Cross-Chain Regulatory Consistency

The central theorem of this theory. Under a valid global state (all chains consistent, no outstanding locks), if a regulatory action is applied via sync, the resulting state is also valid (all chains consistent) and every connected chain reflects the new regulatory state.

This is the regulatory instantiation of the cross-domain consistency theorem from **State_Preservation.thy**.

The proof proceeds by: 1. Unfolding sync into lock, then **update_all_chains**, then unlock 2. Showing **acquire_lock** and **release_lock** preserve chain state 3. Showing **update_all_chains** updates every chain in the target set 4. Combining these facts to establish the conclusion

Auxiliary: update on a single chain preserves consistency for other assets.

lemma *update-chain-preserves-other-consistency*:
assumes *consistent-state* *gs*
and *aid'* $\neq$ *aid*
and *get-reg-state* *gs* *c1* *aid'* = *Some* *s1*
and *get-reg-state* *gs* *c2* *aid'* = *Some* *s2*
shows *get-reg-state* (*update-chain-reg-state* *gs* *cid* *aid* *new-st*) *c1* *aid'* = *Some* *s1*
 $\wedge$ *get-reg-state* (*update-chain-reg-state* *gs* *cid* *aid* *new-st*) *c2* *aid'* = *Some* *s2*
using *assms* unfolding *get-reg-state-def get-asset-state-def*
update-chain-reg-state-def Let-def consistent-state-def
by auto

Auxiliary: **release_lock** does not change chains.

lemma *release-lock-chains*:
gs-chains (*release-lock* *gs* *aid*) = *gs-chains* *gs*
unfolding *release-lock-def* by *simp*

lemma *release-lock-reg-state*:
get-reg-state (*release-lock* *gs* *aid*) *cid* *aid'* = *get-reg-state* *gs* *cid* *aid'*
unfolding *get-reg-state-def get-asset-state-def release-lock-chains* by *simp*

Auxiliary: **acquire_lock** does not change chains.

lemma *acquire-lock-chains*:
assumes *acquire-lock* *gs* *aid* = *Some* *gs'*
shows *gs-chains* *gs'* = *gs-chains* *gs*
using *assms* unfolding *acquire-lock-def is-locked-def*

by (*auto split: if-splits*)

lemma *acquire-lock-reg-state*:

assumes *acquire-lock* *gs aid = Some gs'*
shows *get-reg-state* *gs' cid aid' = get-reg-state* *gs cid aid'*
using *acquire-lock-chains*[*OF* *assms*]
unfolding *get-reg-state-def* *get-asset-state-def* **by** *simp*

Auxiliary: **connected_chains** uses original *gs*, not locked *gs*.

lemma *acquire-lock-asset-exists*:

assumes *acquire-lock* *gs aid = Some gs'*
shows *asset-exists* *gs' cid aid' = asset-exists* *gs cid aid'*
using *acquire-lock-chains*[*OF* *assms*]
unfolding *asset-exists-def* *get-asset-state-def* **by** *simp*

The cross-chain regulatory homomorphism theorem.

theorem *regulatory-homomorphism*:

assumes *valid: valid-state* *gs*
and *exists: asset-exists* *gs source aid*
and *current: get-reg-state* *gs source aid = Some s*
and *trans: reg-transition* *s action = Some s'*
and *synced: sync* *source action aid gs = Some gs'*
and *connected: c ∈ connected-chains* *gs aid*
and *fin: finite* (*connected-chains* *gs aid*)
shows *get-reg-state* *gs' c aid = Some s'*

proof —

— Step 1: **Unfold** *sync* to extract intermediate states

from *synced current trans*

obtain *gs-locked* **where**

lock: acquire-lock *gs aid = Some gs-locked*

and *gs'-def: gs' = release-lock*

(*update-all-chains* *gs-locked aid s' (connected-chains* *gs aid)*) *aid*

unfolding *sync-def*

by (*auto split: option.splits simp: Let-def*)

— Step 2: **acquire_lock** preserves chains

have *locked-chains: gs-chains* *gs-locked = gs-chains* *gs*

using *acquire-lock-chains*[*OF* *lock*] **by** *simp*

— Step 3: *c* is in **connected_chains**, so the asset exists on *c*

from *connected* **have** *asset-exists* *gs c aid*

unfolding *connected-chains-def* **by** *simp*

then obtain *ast-c* **where** *ast-c: gs-chains* *gs c aid = Some ast-c*

unfolding *asset-exists-def* *get-asset-state-def* **by** *auto*

— Step 4: Therefore asset also exists in **gs_locked** on chain *c*

have *ast-locked: gs-chains* *gs-locked c aid = Some ast-c*

using *ast-c locked-chains* **by** *simp*

— Step 5: `update_all_chains` updates chain `c`
have *updated: gs-chains (update-all-chains gs-locked aid s' (connected-chains gs aid))* *c aid =*
 Some (ast-c[as-reg-state := s'])
using *update-all-chains-in-targets[OF connected ast-locked]* **by** *simp*

— Step 6: `release_lock` does not change chains
have *gs-chains gs' c aid = Some (ast-c[as-reg-state := s'])*
using *updated unfolding gs'-def release-lock-chains* **by** *simp*

— Step 7: Extract `reg_state`
then show *?thesis*
 unfolding get-reg-state-def get-asset-state-def **by** *simp*
qed

15 Valid State Preservation

The central safety property: synchronization preserves global state validity.

If the global state is valid (consistent and unlocked) before sync, and sync succeeds, then the resulting global state is also valid.

This closes the inductive invariant: valid initial state + `valid_state` preservation under `sync` = `valid_state` holds at all reachable states.

The proof decomposes into two parts: (1) `consistent_state` preservation: sync updates all connected chains to the same new state and leaves other assets untouched. (2) `no_locked_without_reason` preservation: sync acquires and then releases the lock within the same operation, so the output has no outstanding locks (assuming no other locks were held).

15.1 Part 1: consistent_state preservation

After sync, any two chains holding asset `aid` agree on its new regulatory state (it is `s'`), and any two chains holding a different asset `aid'` still agree (their states are unchanged from the consistent input).

lemma *sync-preserves-consistent-state:*

assumes *valid: valid-state gs*
and *exists: asset-exists gs source aid*
and *current: get-reg-state gs source aid = Some s*
and *trans: reg-transition s action = Some s'*
and *synced: sync source action aid gs = Some gs'*
and *fin: finite (connected-chains gs aid)*
shows *consistent-state gs'*

proof —

— Unfold sync to extract the structure
from *synced current trans*
obtain *gs-locked* **where**
 lock: acquire-lock gs aid = Some gs-locked

```

and gs'-def: gs' = release-lock
      (update-all-chains gs-locked aid s' (connected-chains gs aid)) aid
unfolding sync-def
by (auto split: option.splits simp: Let-def)

have locked-chains: gs-chains gs-locked = gs-chains gs
using acquire-lock-chains[OF lock] by simp

show ?thesis
unfolding consistent-state-def
proof (intro allI impI)
  fix c1 c2 aid' s1-final s2-final
  assume s1-eq: get-reg-state gs' c1 aid' = Some s1-final
    and s2-eq: get-reg-state gs' c2 aid' = Some s2-final

  show s1-final = s2-final
  proof (cases aid' = aid)
    case True
    — Case: the synchronized asset. Two sub-cases per chain.
    show ?thesis
    proof (cases c1 ∈ connected-chains gs aid)
      case c1-in: True
      then have ae1: asset-exists gs c1 aid
        unfolding connected-chains-def by simp
      then obtain ast1 where ast1: gs-chains gs c1 aid = Some ast1
        unfolding asset-exists-def get-asset-state-def by auto
      have ast1-locked: gs-chains gs-locked c1 aid = Some ast1
        using ast1 locked-chains by simp
      have upd1: gs-chains (update-all-chains gs-locked aid s' (connected-chains
gs aid)) c1 aid =
        Some (ast1() as-reg-state := s')
        using update-all-chains-in-targets[OF c1-in ast1-locked] by simp
      have reg1: get-reg-state gs' c1 aid = Some s'
        unfolding gs'-def release-lock-chains get-reg-state-def get-asset-state-def
        using upd1 by simp
      show ?thesis
      proof (cases c2 ∈ connected-chains gs aid)
        case c2-in: True
        then have ae2: asset-exists gs c2 aid
          unfolding connected-chains-def by simp
        then obtain ast2 where ast2: gs-chains gs c2 aid = Some ast2
          unfolding asset-exists-def get-asset-state-def by auto
        have ast2-locked: gs-chains gs-locked c2 aid = Some ast2
          using ast2 locked-chains by simp
        have reg2: get-reg-state gs' c2 aid = Some s'
          unfolding gs'-def release-lock-chains get-reg-state-def get-asset-state-def
          using update-all-chains-in-targets[OF c2-in ast2-locked] by simp
        from reg1 reg2 s1-eq s2-eq True show ?thesis by simp
      next

```

```

case c2-out: False
  — c2 is not connected, so asset doesn't exist on c2 for aid
  have gs-chains (update-all-chains gs-locked aid s' (connected-chains gs
aid)) c2 =
    gs-chains gs-locked c2
    using update-all-chains-outside-targets[OF c2-out] by simp
    then have gs-chains gs' c2 aid = gs-chains gs c2 aid
    unfolding gs'-def release-lock-chains using locked-chains by simp
    moreover have gs-chains gs c2 aid = None
    using c2-out unfolding connected-chains-def asset-exists-def get-asset-state-def
    by auto
    ultimately have get-reg-state gs' c2 aid = None
    unfolding get-reg-state-def get-asset-state-def by simp
    with s2-eq True show ?thesis by simp
qed
next
case c1-out: False
  — c1 not connected: asset doesn't exist there
  have gs-chains (update-all-chains gs-locked aid s' (connected-chains gs aid))
c1 =
    gs-chains gs-locked c1
    using update-all-chains-outside-targets[OF c1-out] by simp
    then have gs-chains gs' c1 aid = gs-chains gs c1 aid
    unfolding gs'-def release-lock-chains using locked-chains by simp
    moreover have gs-chains gs c1 aid = None
    using c1-out unfolding connected-chains-def asset-exists-def get-asset-state-def
    by auto
    ultimately have get-reg-state gs' c1 aid = None
    unfolding get-reg-state-def get-asset-state-def by simp
    with s1-eq True show ?thesis by simp
qed
next
case diff: False
  — Case: a different asset. Sync does not touch it.
  have other1: gs-chains (update-all-chains gs-locked aid s' (connected-chains
gs aid)) c1 aid' =
    gs-chains gs-locked c1 aid'
    using update-all-chains-other-asset[OF diff] by simp
  have other2: gs-chains (update-all-chains gs-locked aid s' (connected-chains
gs aid)) c2 aid' =
    gs-chains gs-locked c2 aid'
    using update-all-chains-other-asset[OF diff] by simp
  have gs-chains gs' c1 aid' = gs-chains gs c1 aid'
  unfolding gs'-def release-lock-chains using other1 locked-chains by simp
  moreover have gs-chains gs' c2 aid' = gs-chains gs c2 aid'
  unfolding gs'-def release-lock-chains using other2 locked-chains by simp
  ultimately have get-reg-state gs' c1 aid' = get-reg-state gs c1 aid'
  and get-reg-state gs' c2 aid' = get-reg-state gs c2 aid'
  unfolding get-reg-state-def get-asset-state-def by simp-all

```



```

with s1-eq s2-eq have
  get-reg-state gs c1 aid' = Some s1-final
  get-reg-state gs c2 aid' = Some s2-final
  by simp-all
with valid show ?thesis
  unfolding valid-state-def consistent-state-def by blast
qed
qed
qed

```

15.2 Part 2: no_locked_without_reason preservation

After sync, no assets are locked. The sync operation acquires a lock on the target asset and releases it before returning. For all other assets, the lock state is unchanged (and was False by `valid_state`).

lemma *sync-preserves-no-locks:*

```

assumes valid: valid-state gs
and exists: asset-exists gs source aid
and current: get-reg-state gs source aid = Some s
and trans: reg-transition s action = Some s'
and synced: sync source action aid gs = Some gs'
shows no-locked-without-reason gs'
proof —
  from synced current trans
obtain gs-locked where
    lock: acquire-lock gs aid = Some gs-locked
and gs'-def: gs' = release-lock
    (update-all-chains gs-locked aid s' (connected-chains gs aid)) aid
unfolding sync-def
by (auto split: option.splits simp: Let-def)

```

— Locks after acquire: only aid is locked

```

from lock have locked-locks: gs-locks gs-locked = (gs-locks gs)(aid := True)
unfolding acquire-lock-def is-locked-def
by (auto split: if-splits)

```

— `update_all_chains` does not change locks

```

have upd-locks: gs-locks (update-all-chains gs-locked aid s' (connected-chains gs aid)) =
  gs-locks gs-locked
using update-all-chains-locks by simp

```

— `release_lock` clears the lock on aid

```

have final-locks: gs-locks gs' = (gs-locks gs-locked)(aid := False)
unfolding gs'-def release-lock-def using upd-locks by simp

```

show *?thesis*

unfolding *no-locked-without-reason-def is-locked-def*

proof

```

fix aid'
show  $\neg$  gs-locks gs' aid'
proof (cases aid' = aid)
  case True
  then show ?thesis using final-locks by simp
next
case False
then have gs-locks gs' aid' = gs-locks gs-locked aid'
  using final-locks by simp
also have ... = gs-locks gs aid'
  using locked-locks False by simp
also have ... = False
using valid unfolding valid-state-def no-locked-without-reason-def is-locked-def
  by simp
finally show ?thesis by simp
qed
qed
qed

```

15.3 The valid_state preservation theorem

Combining the two parts: sync preserves **valid_state**. This is the inductive invariant that guarantees **valid_state** holds at every reachable global state.

theorem *valid-state-preservation:*

```

assumes valid: valid-state gs
  and exists: asset-exists gs source aid
  and current: get-reg-state gs source aid = Some s
  and trans: reg-transition s action = Some s'
  and synced: sync source action aid gs = Some gs'
  and fin: finite (connected-chains gs aid)
shows valid-state gs'
unfolding valid-state-def
using sync-preserves-consistent-state[OF assms]
  sync-preserves-no-locks[OF valid exists current trans synced]
by auto

```

16 Multi-Domain Locale Instantiation

We now instantiate the **multi_domain_preservation** locale from **State_Preservation.thy** with the regulatory state machine.

The instantiation is parametric: given ANY finite set of domains and ANY initial **domain_state** function satisfying the consistency precondition, the locale is satisfied. This proves that the abstract framework applies to the concrete regulatory model without fixing a specific topology.

The key bridge: we construct a **domain_state** function from our **global_state**, and show that **valid_state** implies the consistency precondition required by **multi_domain_preservation**.

Bridge from our `global_state` model to the `multi_domain_preservation` locale's `domain_state` signature.

`multi_domain_preservation` expects: `domain_state :: 'd => 'id => 's option`

We instantiate with: `'d = chain_id, 'id = asset_id, 's = reg_state`

The bridge function extracts `reg_state` from our richer `asset_state`.

definition *reg-domain-state* :: *global-state* $\Rightarrow$ *chain-id* $\Rightarrow$ *asset-id* $\Rightarrow$ *reg-state option* **where**

reg-domain-state *gs cid aid* = *get-reg-state* *gs cid aid*

The parametric instantiation theorem: for any finite set of `chain_ids` and any valid `global_state`, we can interpret the `multi_domain_preservation` locale with the regulatory state machine.

The `state_machine` predicate as a standalone fact. This is needed for instantiating `multi_domain_preservation`, which takes `state_machine` as an opaque predicate assumption.

lemma *reg-state-machine-pred*:

state-machine *reg-states* *reg-actions* *reg-transition* *reg-terminal*

proof –

have *f1*: *finite* *reg-states* **unfolding** *reg-states-def* **by** *auto*

have *f2*: *finite* *reg-actions* **unfolding** *reg-actions-def* **by** *auto*

have *f3*: *reg-terminal* $\subseteq$ *reg-states*

unfolding *reg-terminal-def* *reg-states-def* **by** *auto*

have *f4*: $\bigwedge s\ a.\ s \in \text{reg-terminal} \implies a \in \text{reg-actions} \implies \text{reg-transition } s\ a = \text{None}$

unfolding *reg-terminal-def* **by** (*auto simp: confiscated-terminal*)

have *f5*: $\bigwedge s\ a\ s'.\ s \in \text{reg-states} \implies a \in \text{reg-actions} \implies \text{reg-transition } s\ a = \text{Some } s' \implies s' \in \text{reg-states}$

using *reg-transition-closed* **by** *auto*

have *f6*: $\bigwedge s\ a.\ (s :: \text{reg-state}) \notin \text{reg-states} \implies \text{reg-transition } s\ a = \text{None}$

using *reg-states-UNIV* **by** *auto*

show *?thesis*

by (*intro state-machine.intro*) (*use f1 f2 f3 f4 f5 f6 in auto*)

qed

theorem *reg-multi-domain-instantiation*:

assumes *fin-doms*: *finite* (*doms* :: *chain-id* *set*)

and *valid*: *valid-state* *gs*

shows *multi-domain-preservation* *doms* *reg-states* *reg-actions* *reg-transition* *reg-terminal* (*reg-domain-state* *gs*)

proof (*intro multi-domain-preservation.intro*)

show *finite* *doms* **using** *fin-doms* **by** *simp*

next

show *state-machine* *reg-states* *reg-actions* *reg-transition* *reg-terminal*

by (*rule reg-state-machine-pred*)

next

fix *d1 d2* :: *chain-id* **and** *aid* :: *asset-id* **and** *s1 s2* :: *reg-state*

```

assume  $d1 \in \text{doms}$   $d2 \in \text{doms}$ 
and  $\text{reg-domain-state } gs \ d1 \ \text{aid} = \text{Some } s1$ 
and  $\text{reg-domain-state } gs \ d2 \ \text{aid} = \text{Some } s2$ 
then have  $\text{get-reg-state } gs \ d1 \ \text{aid} = \text{Some } s1$ 
and  $\text{get-reg-state } gs \ d2 \ \text{aid} = \text{Some } s2$ 
unfolding  $\text{reg-domain-state-def}$  by  $\text{simp-all}$ 
with  $\text{valid}$  show  $s1 = s2$ 
unfolding  $\text{valid-state-def}$   $\text{consistent-state-def}$  by  $\text{blast}$ 
qed

```

Corollary: the generic `cross_domain_consistency` theorem from `State_Preservation.thy` applies to regulatory synchronization.

This connects the abstract theory to the concrete model: the generic theorem guarantees that after `sync_all` (the abstract synchronization), all connected domains agree. Combined with our concrete `regulatory_homomorphism` theorem, we have both the abstract and concrete guarantees.

The instantiation above (`reg_multi_domain_instantiation`) enables using all theorems proven in `multi_domain_preservation` in particular `cross_domain_consistency` and `sync_isolation` for the regulatory model without reproving them. Any consumer of this theory can invoke:

```

multi_domain_preservation.cross_domain_consistency [OF reg_multi_domain_instantiation
fin_doms valid_gs]]

```

to obtain the concrete guarantee for their domain set and global state.

Summary of what this theory establishes:

1. The regulatory state machine (`reg_state`, `reg_action`, `reg_transition`) satisfies the `state_machine` locale from `State_Preservation.thy`.

2. CONFISCATED is terminal (I1), CONFISCATE is universally reachable (I2), and the transition function is deterministic (I3).

3. SEIZED to FROZEN and FROZEN to RESTRICTED direct transitions are excluded by design, with legal and complexity-reduction justifications.

4. The synchronization protocol (lock, then update, then unlock) preserves cross-chain consistency for the same asset while isolating other assets.

5. Preemptive locking prevents concurrent regulatory actions on the same asset (NOT double-spend prevention that is handled at a different layer).

6. The `regulatory_homomorphism` theorem establishes that after `sync`, all connected chains agree on the new regulatory state.

7. The `valid_state_preservation` theorem establishes that `sync` preserves the global validity invariant: `consistent_state` AND no outstanding locks. This closes the inductive invariant.

8. The `multi_domain_preservation` locale is instantiated parametrically: for any finite domain set and valid global state, the abstract framework from `State_Preservation.thy` applies to the regulatory model.

What remains for future work: - Prove eventual consistency under the

partially synchronous network model (with message delays, redelivery, and out-of-order arrival) - (Optional) Connect to the `state_preservation` locale with identity state/action maps for the uniform-model case
end

17 Acknowledgments

The regulatory state model (five states, seven actions, and the transition exclusions) is motivated by the Regulatory Compliance Protocol (RCP), a framework developed by the Oraclizer research team that systematically classifies requirements from 15 global financial regulatory authorities for tokenized capital markets. RCP is currently proposed as an Informational Ethereum Improvement Proposal (EIP) in Draft status.